STACK

Definition: It is a non-primitive linear data structure into which a new element can be added or from which an element can be deleted at only one end called the *top of the stack*. The other end is called the *bottom of the stack*.

Stack is also called as **LIFO** (Last In First Out) data structure since the last element inserted will be the first to be removed from the stack.

Representation of Stack in C:

```
#define MAXSIZE 4
typedef struct
{
        int items[MAXSIZE];
        int top;
}STACK;
```

Basic Operations on Stack:

- Push operation
- Pop operation
- Peep/peek operation
- Display operation

PUSH Operation on Stack:

- Inserting a new element onto the top of the stack is referred to as **Push** operation.
- If the stack is full and an attempt is made to insert an element onto the stack then it results in a situation called "**Stack Overflow**" !!!
- Condition to test for "Stack Overflow" : **if(top == MAXSIZE-1)**
  If this condition is true then it indicates that ***stack is full***.

- Assume, MAXSIZE is 3 and the data to be inserted are 10, 5, 3, 7

Figure1: Initial Stack (Empty Stack).

Figure2: After inserting 10.

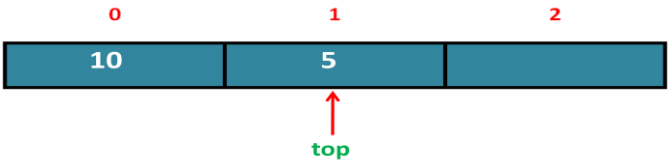


Figure3: After inserting 5.

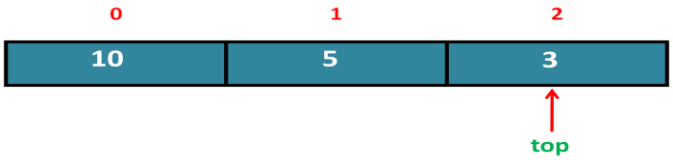


Figure4: After inserting 3.

POP Operation on Stack:

- Deleting an element from the top of the stack is referred to as **Pop** operation.
- If the stack is empty and an attempt is made to delete an element from the stack then it results in a situation called "**Stack underflow**"!!!
- Condition to test for "Stack underflow" : **if(top == -1)**
  If this condition is true then it indicates that ***stack is empty***.

- Assume, MAXSIZE is 3

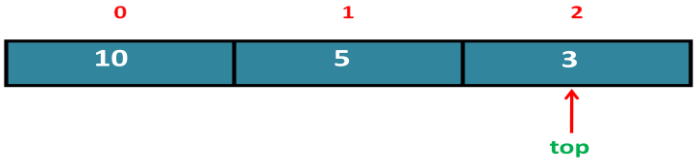


Figure1: Current Stack Status .

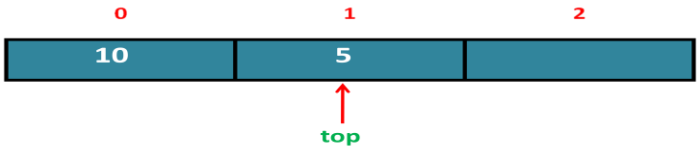


Figure2: After deleting 3.

Figure3: After deleting 5.

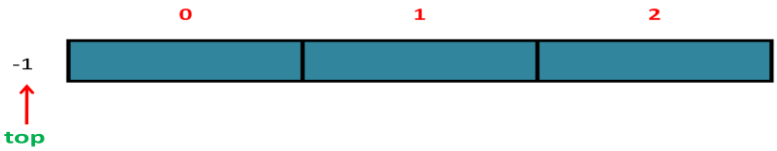
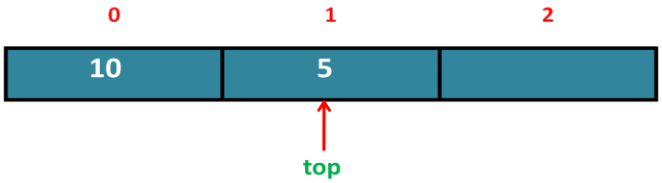


Figure4: After deleting 10.

PEEP/PEEK Operation on Stack:

- Retrieving the topmost element from the stack is referred to as Peep/Peek operation.



Topmost element retrieved is 5 !!!

Display Operation on Stack:

- Display operation means listing the contents of the stack either from bottom of the stack till top or from top of the stack till bottom of the stack.
- Check for stack empty: **if(top==-1) printf("Stack is Empty");**
- Print the stack elements one by one:

```
for(i=0;i<=top;i++)
    printf("%d->",items[i]);
```

Program: ***Develop a C program to implement stack of integers to perform push, pop, peek and display operations.***

```
#include<stdio.h>
#include<stdlib.h>
#define MAXSIZE  3

typedef struct
{
       int items[MAXSIZE];
       int top;
}STACK;

int isfull(STACK s)
{
      if(s.top==MAXSIZE-1)
          return 1;
       return 0;
}

int isempty(STACK s)
{
      if(s.top==-1)
           return 1;
      return 0;
 }

void PUSH(STACK *s,int data)
{
        s->items[++s->top]=data;
        printf("\n%d is pushed onto stack",data);
}

int POP(STACK *s)
{
         return(s->items[s->top--]);
}

int PEEK(STACK s)
{
       return(s.items[s.top]);
}

void DISPLAY(STACK s)
{
       int i;

       printf("\nSTACK CONTENTS:\nBOS->");
       for(i=0;i<=s.top;i++)
```

```
            printf("%d->",s.items[i]);
        printf("TOS");
}

int main()
{
    STACK s;
    int data,choice;
    s.top = -1;
    while(1)
    {
        printf("\n\n1:Push\n2:Pop\n3:Peek\n4:Display\n5:Exit");
        printf("\nEnter your choice: ");
        scanf("%d",&choice);
        switch(choice)
        {
            case 1: if(isfull(s))
                        printf("\nSTACK OVERFLOW");
                    else
                    {
                        printf("\nEnter the data to be pushed: ");
                        scanf("%d",&data);
                        PUSH(&s,data);
                    }
                    break;
             case 2:  if(isempty(s))
                        printf("\nSTACK UNDERFLOW");
                      else
                        printf("\n%d is popped from top of the stack",POP(&s));
                      break;

             case 3:  if(isempty(s))
                        printf("\nSTACK EMPTY");
                      else
                        PEEK(s);
                      break;

            case 4: if(isempty(s))
                        printf("\nSTACK EMPTY");
                    else
                        DISPLAY(s);
                    break;

             case 5:exit(0);
            default: printf("\nInvalid choice");
        }
    }
    return 0;
}
```

Lab Program2: ***Develop a C program to implement Stack of names to perform the push, pop and display operations.***

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#define MAXSIZE 3
 typedef struct
{
        char items[MAXSIZE][25];
        int top;
}STACK;

int isfull(STACK s)
{
        if(s.top==MAXSIZE-1)
           return 1;
        return 0;
}

int isempty(STACK s)
{
         if(s.top==-1)
              return 1;
          return 0;
}

void PUSH(STACK *s,char name[])
{
         strcpy(s->items[++s->top],name);
         printf("\nName %s is pushed on to the stack",name);
}

char* POP(STACK *s)
{
        return(s->items[s->top--]);
}

void DISPLAY(STACK s)
{
         int i;
         printf("\nSTACK CONTENTS:\nBOS->");
        for(i=0;i<=s.top;i++)
               printf("%s->",s.items[i]);
        printf("TOS");
}
```

```
int main()
{
      STACK s;
      int choice;
      char name[20];

      s.top = -1;

      while(1)
     {
           printf("\n\n1:Push\n2:Pop\n3:Display\n4:Exit");
           printf("\nEnter your choice: ");
           scanf("%d",&choice);
           switch(choice)
          {
                case 1: if(isfull(s))
                              printf("\nSTACK OVERFLOW");
                         else
                        {
                              printf("\nEnter the name to be pushed: ");
                              scanf("%s",name);
                              PUSH(&s,name);
                         }
                         break;


                case 2: if(isempty(s))
                              printf("\nSTACK UNDERFLOW");
                          else
                              printf("\nName %s is popped from top of the stack",POP(&s));
                          break;

                case 3: if(isempty(s))
                              printf("\nSTACK EMPTY");
                         else
                              DISPLAY(s);
                          break;
                case 4: exit(0);

                default: printf("\nInvalid choice");
              }
       }
       return 0;
}
```

Expressions

- Sequence of operands and operators that reduces to a single value after evaluation is referred to as an expression.
- Operands can be either a constant or a variable.
- Operators can be +, -, *, /, % and $ or ^.

Different forms of expressions:

Infix expression:

- An expression in which the operator is in between the two operands is referred to as an Infix expression.
- Infix expression can be either parenthesized or unparenthesized.
- **Example:** a+b

Prefix expression/Polish expression:

- An expression in which the operator precedes the two operands is referred to as Prefix expression.
- Prefix expression is always unparenthesized.
- **Example:** +ab

Postfix expression/Reverse Polish expression/Suffix expression:

- An expression in which the operator follows the two operands is referred to as Postfix expression.
- Postfix expression is always unparenthesized.
- **Example:** ab+

Problems:

Convert the following Infix expressions into its equivalent Prefix and Postfix expressions:

1. a + b

Prefix form:	Postfix form:
+ a b	a b +

2. a + b * c

Prefix form:

a + b * c
a + *b c
+ a *b c

Postfix form:

a + b * c
a + b c *
a b c * +

3. a * (b + c)

Prefix form:

a * (b + c)
a * + b c
* a + b c

Postfix form:

a * (b + c)
a * b c +
a b c + *

4. (A + B) * (C - D)

Prefix form:

(A + B) * (C – D)
+ A B * (C – D)
+ A B * - C D
* + A B – C D

Postfix form:

(A + B) * (C – D)
A B + * (C – D)
A B + * C D -
A B + C D - *

5. 2 $ 3 $ 2

Prefix form:

2 $ 3 $ 2
2 $ $ 3 2
$ 2 $ 3 2

Postfix form:

2 $ 3 $ 2
2 $ 3 2 $
2 3 2 $ $

6. a + ((b + c) * d)

Prefix form:

a + ((b + c) * d)
a + (+ b c * d)
a + * + b c d
+ a * + b c d

Postfix form:

a + ((b + c) * d)
a + (b c + * d)
a + b c + d *
a b c + d * +

7. A $ B * C – D + E / F / (G + H)

Prefix form:

A $ B * C – D + E / F / (G + H)
A $ B * C – D + E / F / +GH
$AB * C – D + E / F / +GH
*$ABC – D + E / F / +GH
*$ABC – D + /EF / +GH
*$ABC – D + //EF+GH
-*$ABCD + //EF+GH
+ - * $ A B C D / / E F + G H

Postfix form:

A $ B * C – D + E / F / (G + H)
A $ B * C – D + E / F / GH+
AB$ * C – D + E / F / GH+
AB$C* – D + E / F / GH+
AB$C* – D + EF/ / GH+
AB$C* – D + EF/GH+/
AB$C*D- + EF/GH+/
A B $ C * D – E F / G H + / +

8. ((A + (B – C) * D) ^ E + F)

Prefix form:

((A + (B – C) * D) ^ E + F)
((A + -BC * D) ^ E + F)
((A + *-BCD) ^ E + F)
(+A*-BCD ^ E + F)
(^+A*-BCDE + F)
+ ^ + A * - B C D E F

Postfix form:

((A + (B – C) * D) ^ E + F)
((A + BC- * D) ^ E + F)
((A + BC-D*) ^ E + F)
(ABC-D*+ ^ E + F)
(ABC-D*+E^ + F)
A B C – D * + E ^ F +

Applications of Stack

- Conversion of expression from one form to another
- Evaluation of Prefix/Postfix expression
- Recursion
- Checking for string palindrome
- Checking for validity of expression

Evaluation of Postfix expression

Procedure/Algorithm:

1. Scan each symbol in the postfix expression from left to right and perform one of the following operations till end of input is encountered:
   a) If scanned symbol is an operand then push it on to the stack.
   b) If the scanned symbol is an operator then
      i. Pop top two elements from the stack. First popped is operand2 and second popped is operand1
      ii. Perform the indicated operation
          result = operand1 operator operand2
      iii. Push the result back to the stack.
2. Pop the result of evaluation from the top of the stack.

Tracing:

1. 2 3 +

Symbol	operand1	operand2	result	Stack
2				2
3				2,3
+	2	3	2+3=5	5

2. 3 5 + 4 * 2 /

Symbol	operand1	operand2	result	Stack
3				3
5				3,5
+	3	5	3+5=8	8
4				8,4
*	8	4	8*4=32	32
2				32,2
/	32	2	32/2=16	16

Lab Program4: ***Develop a C program to evaluate the given postfix expression.***

```
#include<stdio.h>
#include<ctype.h>
#include<math.h>
#define MAXSIZE 20
 typedef struct
{
        float items[MAXSIZE];
        int top;
}STACK;

void PUSH(STACK *s,float data)
{
         s->items[++s->top] = data;
}

 float POP(STACK *s)
{
        return(s->items[s->top--]);
}

float compute(float op1,char symb,float op2)
{
          switch(symb)
         {
                case '+': return op1+op2;
                case '-':  return op1-op2;
```

```
            case '*': return op1*op2;
            case '/': return op1/op2;
            case '$':
            case '^': return pow(op1,op2);
        }
}

int main()
{
        STACK s;
        char postfix[30],symb;
        float n1,n2,res,data;
        int i;

        s.top=-1;

        printf("\nEnter a valid postfix expression:\n");
        scanf("%s",postfix);

        for(i=0;postfix[i]!='\0';i++)
        {
             symb=postfix[i];
              if(isdigit(symb))
                  PUSH(&s,symb-'0');
             else if(isalpha(symb))
            {
                  printf("\n%c = ",symb);
                  scanf("%f",&data);
                  PUSH(&s,data);
            }
            else
            {
                   n2=POP(&s);
                   n1=POP(&s);
                   res=compute(n1,symb,n2);
                   PUSH(&s,res);
               }
          }

          printf("\nResult of evaluation: %f",POP(&s));

          return 0;
}
```

Evaluation of Prefix expression

Procedure/Algorithm:

1. Scan each symbol in the prefix expression from right to left and perform one of the following operations till end of input is encountered:
   a) If scanned symbol is an operand then push it on to the stack.
   b) If the scanned symbol is an operator then
      i. Pop top two elements from the stack. First popped is operand1 and second popped is operand2
      ii. Perform the indicated operation
          result = operand1 operator operand2
      iii. Push the result back to the stack.
2. Pop the result of evaluation from the top of the stack.

Tracing:

1. + 2 3

Symbol	operand1	operand2	result	Stack
3				3
2				3,2
+	2	3	2+3=5	5

2. * 2 – 5 1

Symbol	operand1	operand2	result	Stack
1				1
5				1,5
-	5	1	5-1=4	4
2				4,2
*	2	4	2*4=8	8

Program: ***Develop a C program to evaluate the given prefix expression.***

```
#include<stdio.h>
#include<string.h>
#include<math.h>
#include<ctype.h>
#define MAXSIZE 20

typedef struct
{
        float items[MAXSIZE];
        int top;
}STACK;

void PUSH(STACK *s,float data)
{
         s->items[++s->top] = data;
}

float POP(STACK *s)
{
        return(s->items[s->top--]);
}

float compute(float op1,char symb,float op2)
{
          switch(symb)
         {
                case '+': return op1+op2;
                case '-':  return op1-op2;
                case '*': return op1*op2;
                case '/': return op1/op2;
                case '$':
                case '^': return pow(op1,op2);
         }
}

int main()
{
        STACK s;
        char prefix[30],symb;
        float n1,n2,res,data;
        int i;

        s.top=-1;

        printf("\nEnter a valid prefix expression:\n");
        scanf("%s",prefix);

        for(i=strlen(prefix)-1;i>=0;i--)
        {
                symb=prefix[i];
                if(isdigit(symb))
                        PUSH(&s,symb-'0');
```

```
            else if(isalpha(symb))
            {
                printf("\n%c = ",symb);
                scanf("%f",&data);
                PUSH(&s,data);
            }
            else
            {
                n1=POP(&s);
                n2=POP(&s);
                res=compute(n1,symb,n2);
                PUSH(&s,res);
            }
        }

        printf("\nResult of evaluation: %f",POP(&s));
        return 0;
}
```

Conversion of Infix expression to Postfix

Procedure/Algorithm:

1. **Push '#' onto the stack.**
2. **Scan each symbol in the infix expression from left to right and perform one of the following operations till end of input is encountered:**
   a) **If the scanned symbol is an operand then, add it to the postfix expression.**
   b) **If the scanned symbol is '(' (left parenthesis) then, push it onto the stack.**
   c) **If the scanned symbol is ')' (right parenthesis) then repeatedly pop each operator from the top of the stack and add it to the postfix expression till left parenthesis is encountered. Pop left parenthesis but don't add it to postfix expression.**
   d) **If the scanned symbol is an operator then check the precedence of scanned operator with top of the stack operator.**
      i. **Repeatedly pop each operator from the stack and add it to postfix expression if scanned operator has lower or same precedence as that of top of the stack operator.**
      ii. **Push the scanned operator onto the stack.**
3. **Pop the remaining operators one by one from the stack and add them to the postfix expression.**

Tracing:

1. a + b

Symbol	Stack	Postfix
a	#	a
+	#+	a
b	#+	ab+

2. a * (b+c)

Symbol	Stack	Postfix
a	#	a
*	#*	a
(	#*(	a
b	#*(	ab
+	#*(+	ab
c	#*(+	abc
)	#*(+	abc+*

Lab Program3: ***Develop a C program to convert infix expression to postfix.***

```
#include<stdio.h>
#include<ctype.h>
#define MAXSIZE 25
typedef struct
{
        char items[MAXSIZE];
        int top;
}STACK;

void PUSH(STACK *s,char data)
{
         s->items[++s->top] = data;
}
```

```
char POP(STACK *s)
{
        return(s->items[s->top--]);
}

char PEEK(STACK s)
{
        return(s.items[s.top]);
}

int preced(char symb)
{
       switch(symb)
     {
          case '#':
          case '(': return 0;

          case '+':
          case '-': return 1;

          case '*':
          case '/':
          case '%': return 2;

          case '$':
          case '^': return 3;
        }
}

int main()
{
        STACK s;
        char infix[30],postfix[30],symb,ch;
        int i,j=0;

        s.top=-1;

        printf("\nEnter a valid infix expression:\n");
        scanf("%s",infix);

        PUSH(&s,'#');
        for(i=0;infix[i]!='\0';i++)
       {
              symb=infix[i];

              if(isalnum(symb))
                    postfix[j++]=symb;
              else
             {
```

```
                switch(symb)
               {
                     case '(': PUSH(&s,'(');
                               break;

                     case ')': while((ch=POP(&s))!='(')
                                     postfix[j++]=ch;
                               break;

                    default: while(preced(symb)<=preced(PEEK(s)))
                             {
                                   if(symb==PEEK(s) && preced(symb)==3)
                                        break;
                                   postfix[j++]  = POP(&s);
                              }
                             PUSH(&s,symb);
                }
          }

          while(PEEK(s)!='#')
               postfix[j++]=POP(&s);
          postfix[j]='\0';

          printf("\nResultant Postfix Expression:\n");
          printf("%s",postfix);
          return 0;
}
```

Tracing:

3. 2 $ 3 $ 2

Symbol	Stack	Postfix
2	#	2
$	#$	2
3	#$	23
$	#$$	23
2	#$$	232$$

Conversion of Infix expression to Prefix

Procedure/Algorithm:

1. Push '#' onto the stack.
2. Scan each symbol in the infix expression from right to left and perform one of the following operations till end of input is encountered:
   a) If the scanned symbol is an operand then, add it to the prefix expression.
   b) If the scanned symbol is ')' (right parenthesis) then, push it onto the stack.
   c) If the scanned symbol is '(' (left parenthesis) then repeatedly pop each operator from the top of the stack and add it to the prefix expression till right parenthesis is encountered. Pop right parenthesis but don't add it to prefix expression.
   d) If the scanned symbol is an operator then check the precedence of scanned operator with top of the stack operator.
      i. Repeatedly pop each operator from the stack and add it to prefix expression if scanned operator has lower precedence as that of top of the stack operator.
      ii. Push the scanned operator onto the stack.
3. Pop the remaining operators one by one from the stack and add them to the prefix expression.
4. Reverse the prefix expression to get the final prefix expression.

Tracing:

1. a + b

Symbol	Stack	Prefix
b	#	b
+	#+	b
a	#+	ba+

Final Prefix expression is +ab

Tracing:

2. a * (b+c)

Symbol	Stack	Prefix
)	#)	
c	#)	c
+	#)+	c
b	#)+	cb
(	#)+	cb+
*	#*	cb+
a	#*	cb+a*

Final Prefix expression is *a+bc

Recursion

Definition: The process in which a function calls itself directly or indirectly is referred to as **recursion** and the corresponding function is called as a **recursive function**.

Properties of a recursive function

- It should have at least one base case that doesn't involve call to itself.
- Each time a recursive function is called, the arguments passed must be updated so that it comes closer to the base case.

Differences between Iteration and Recursion

Iteration	Recursion
It uses looping control constructs such as while, do while and for.	It uses conditional constructs such as if, if-else, switch.
It terminates when loop condition fails.	It terminates when the base case is reached.
It becomes infinite when loop condition never fails.	It becomes infinite when there is no base case or base case is never reached.
It consumes less memory space and executes faster.	It takes more time and consumes more memory space.
It is not suitable for applications such as Towers of Hanoi, Tree traversals etc.	It is best suitable for applications such as Towers of Hanoi, Tree traversals etc.
Tracing and debugging is easy	Tracing and debugging is difficult

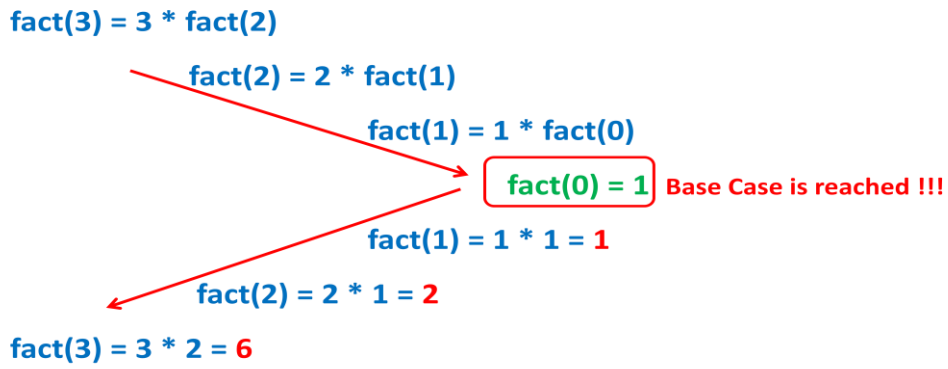
Problem1: To find the factorial of a number.

Recursive Definition

$$fact(n) = \begin{cases} 1 & \text{if } n = 0 \\ n*fact(n-1) & \text{otherwise} \end{cases}$$

```
//Recursive Function
int fact(int n)
{
        if(n==0)        // Base Case
            return 1;

        return n*fact(n-1); //General Case
}
```

Tracing:



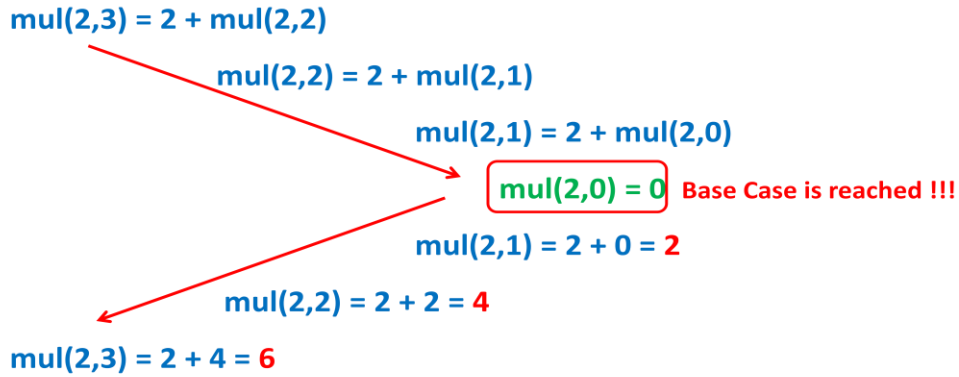
Problem2: To compute product of two positive integers.

Recursive Definition

$$mul(a,b) = \begin{cases} 0 & \text{if } a = 0 \text{ or } b=0 \\ a+mul(a,b-1) & \text{otherwise} \end{cases}$$

```
//Recursive Function
int mul(int a,int b)
{
        if(a == 0 || b == 0)  // Base Case
              return 0;
         return(a + mul(a,b-1)); //General Case
}
```

Tracing:



Problem3: To compute sum of first *n* natural numbers.

Recursive Definition

$$sum(n) = \begin{cases} 1 & \text{if } n = 1 \\ n+sum(n-1) & \text{otherwise} \end{cases}$$

```
//Recursive Function
int sum(int n)
{
        if(n == 1)  // Base Case
            return 1;

        return(n + sum(n-1)); //General Case
}
```

Tracing:

sum(4) = 4 + sum(3)

sum(3) = 3 + sum(2)

sum(2) = 2 + sum(1)

sum(1) = 1 Base Case is reached !!!

sum(2) = 2 + 1 = 3

sum(3) = 3 + 3 = 6

sum(4) = 4 + 6 = 10

Problem4: To compute sum of squares of first *n* natural numbers.

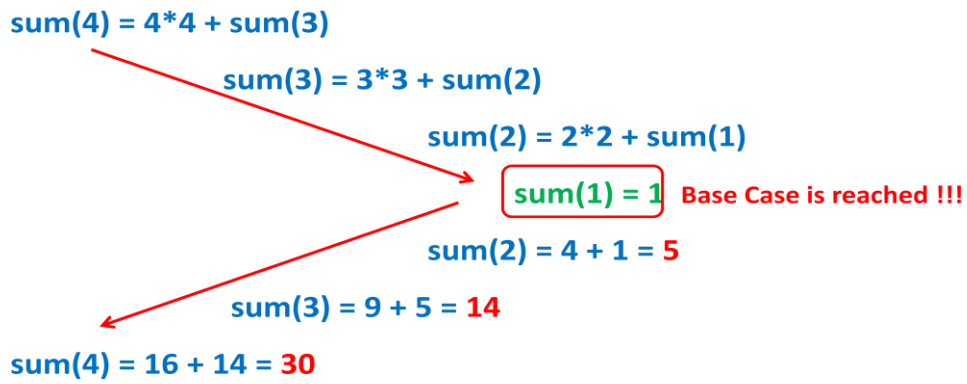
Recursive Definition

$$sum(n) = \begin{cases} 1 & \text{if } n = 1 \\ n*n+sum(n-1) & \text{otherwise} \end{cases}$$

```
//Recursive Function
int sum(int n)
{
        if(n == 1)  // Base Case
            return 1;
```

```
        return(n*n+sum(n-1)); //General Case
}
```

Tracing:



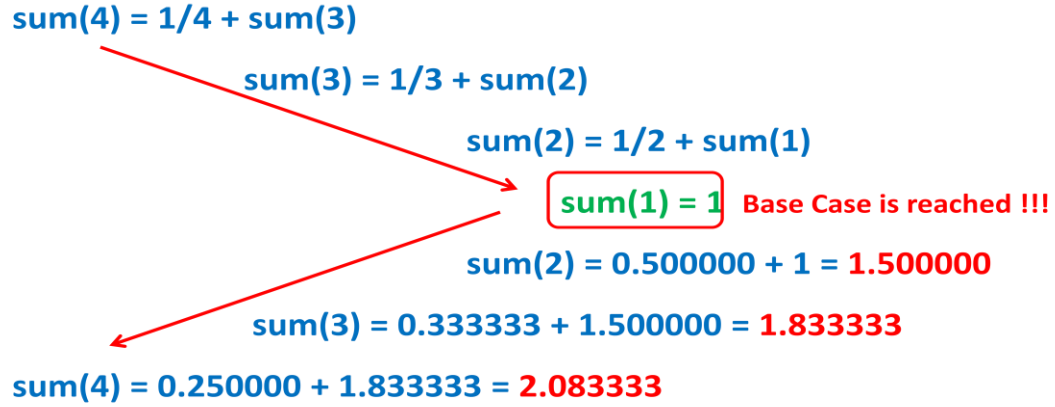
Problem5: To compute sum of the series 1 + 1/2 +1/3+ . . . +1/n

Recursive Definition

$$sum(n) = \begin{cases} 1 & \text{if } n = 1 \\ 1/n+sum(n-1) & \text{otherwise} \end{cases}$$

```
//Recursive Function
float sum(int n)
{
    if(n == 1)  // Base Case
        return 1;

    return(1.0/n + sum(n-1)); //General Case
}
```

Tracing:



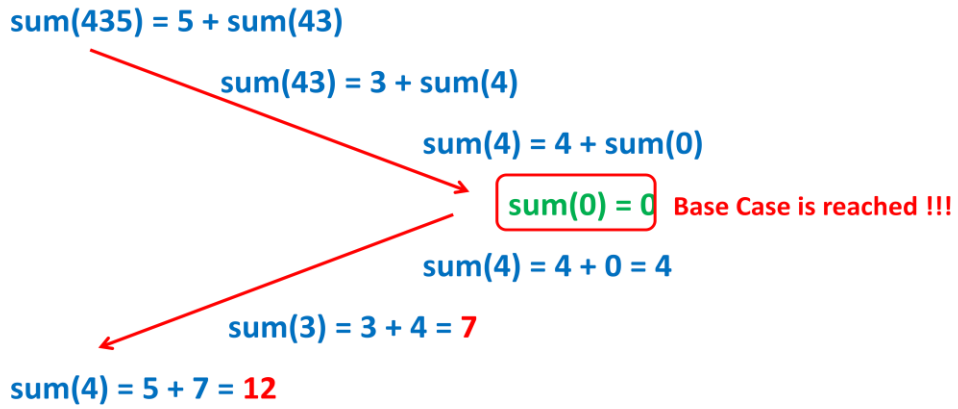
Problem6: To compute sum of all the digits of a positive integer

Recursive Definition

$$sum(n) = \begin{cases} 0 & \text{if } n = 0 \\ n\%10+sum(n/10) & \text{otherwise} \end{cases}$$

```
//Recursive Function
int sum(int n)
{
      if(n == 0)  // Base Case
          return 0;

      return(n%10 + sum(n/10)); //General Case
}
```

Tracing:



Problem7: To compute x^n

```
//Recursive Function

float find(int x,int n)
{
      if(n == 0)  // Base Case
           return 1;
      if(n>0)
           return(x * find(x,n-1)); //General case
       return(1.0/x * find(x,n+1));
}
```

Tracing1:

find(2,3) = 2 * find(2,2)

find(2,2) = 2 * find(2,1)

find(2,1) = 2 * find(2,0)

find(2,0) = 1 Base Case is reached !!!

find(2,1) = 2 * 1 = 2

find(2,2) = 2 * 2 = 4

find(2,3) = 2 * 4 = 8

Tracing2:

find(2,-3) = 1/2 * find(2,-2)

find(2,-2) = 1/2 * find(2,-1)

find(2,-1) = 1/2 * find(2,0)

find(2,0) = 1 Base Case is reached !!!

find(2,-1) = 1/2 * 1 = 0.500000

find(2,-2) = 1/2 * 0.500000 = 0.250000

find(2,-3) = 1/2 * 0.250000 = 0.125000

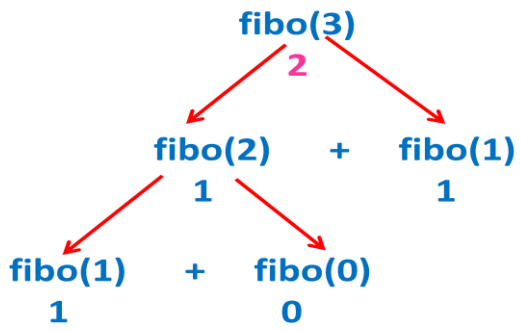
Problem8: To find the nth Fibonacci number (0 1 1 2 3 5....)

Recursive Definition

$$fibo(n) = \begin{cases} n & \text{if } n = 0 \text{ or } n=1 \\ fibo(n-1)+fibo(n-2) & \text{otherwise} \end{cases}$$

```
//Recursive Function
int fibo(int n)
{
    if(n == 0 || n==1)  // Base Case
        return n;

    return(fibo(n-1)+fibo(n-2)); //General case
}
```

Tracing:



Program: Develop a C Program to generate first *n* Fibonacci numbers. Use recursive C function to find *n*th Fibonacci number.

```
#include<stdio.h>
int fibo(int n)
{
      if(n == 0 || n == 1)  // Base Case
          return n;

      return(fibo(n-1) + fibo(n-2)); //General case
}

int main()
{
        int i,n;

        printf("\nEnter the value of n: ");
        scanf("%d",&n);
        printf("\nFirst %d Fibonacci numbers:\n",n);
        for(i=0;i<n;i++)
              printf("%d ",fibo(i));

        return 0;
}
```

Problem9: To compute GCD of two integers.

Recursive Definition

$$gcd(m,n) = \begin{cases} m & \text{if } n = 0 \\ gcd(n, m\%n) & \text{otherwise} \end{cases}$$

```
//Recursive Function
int gcd(int m,int n)
{
      if(n == 0)  // Base Case
            return m;

      return(gcd(n,m%n)); //General Case
}
```

Tracing:

```
gcd(35,15)
      gcd(15,5)
            gcd(5,0)
                  return 5
```

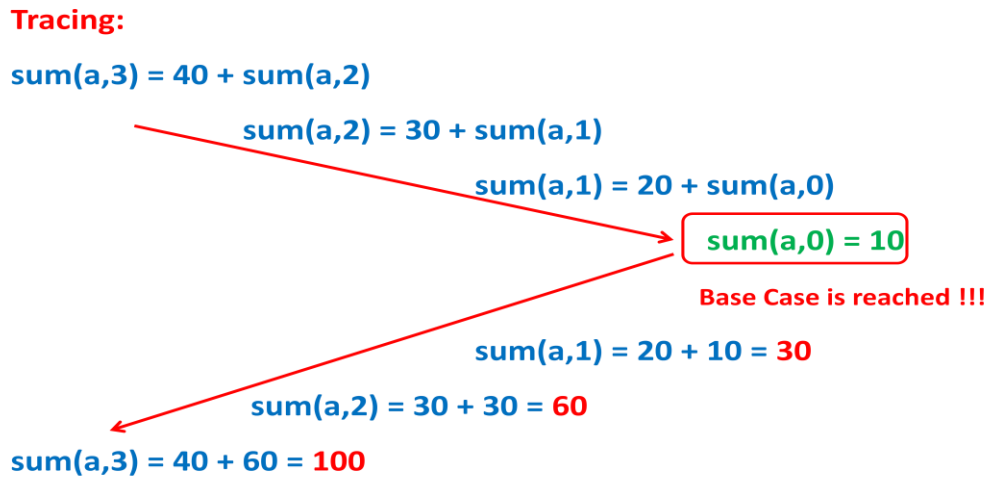
Problem10: To compute sum of all integers in an array of size *n*.

In main() function:

```
printf("\nSum = %d",sum(a,n-1));
```

```
//Recursive Function
int sum(int a[],int n)
{
      if(n == 0)  // Base Case
          return a[n];
      return(a[n] + sum(a,n-1)); //General Case
}
```

a[0]	a[1]	a[2]	a[3]
10	20	30	40



Problem11: To print the array elements in reverse order.

In main() function:
print(a,0,4);

```
void print(int a[],int i,int n)
{
      if(i == n)          //Base Case
          return;
      print(a,i+1,n);  //General Case
      printf("%d ",a[i]);
}
```

Problem12: To search for the key element using Binary search technique.

In main() function:
res = search(a,0,n-1,key);

```
//Recursive Function
int search(int a[],int low,int high,int key)
{
        int mid;

        if(low>high)    //Base case for failure
             return -1;
        mid=(low+high)/2;
         if(key == a[mid])     // Base case for success
               return(mid+1);
         if(key<a[mid])
```

```
        return(search(a,low,mid-1,key));

    return(search(a,mid+1,high,key);
}
```

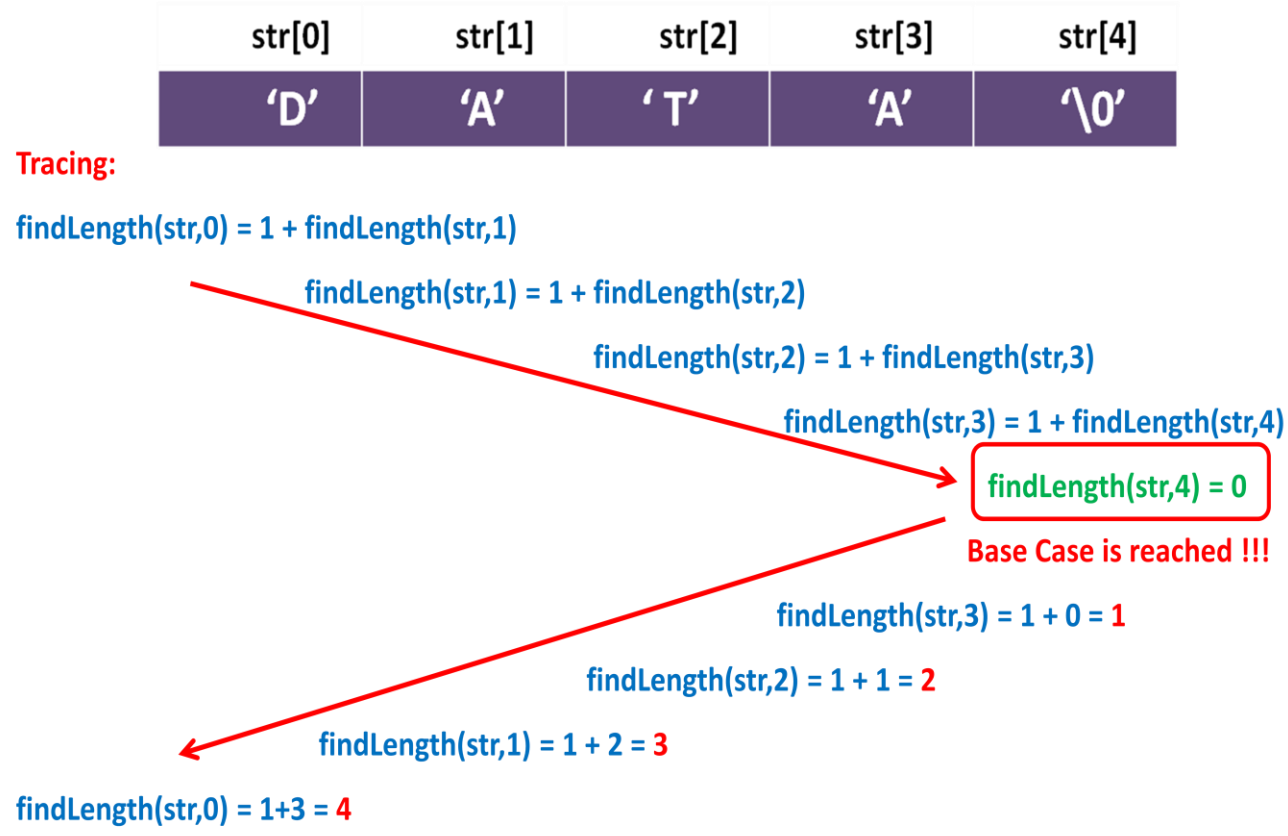
Problem13: To find the length of the string.

In main() function:

```
printf("\nLength = %d",findLength(str,0));
```

//Recursive Function

```
int findLength(char  str[],int i)
{
    if(str[i]=='\0')  //Base Case
        return 0;
    return(1+findLength(str,i+1));   //General Case
}
```



Problem14: To search for a character in a string.

In main() function:

```
res = search(str,0,ch);
```

//Recursive Function

```
int search(char  str[],int i,char ch)
{
        if(str[i]=='\0')  //Base Case for failure
                return(-1);
        if(str[i] == ch)   //Base Case for success
                return(i+1);
        return(search(str,i+1,ch));   //General Case
}
```

Problem15: To check whether a given string is a palindrome or not.

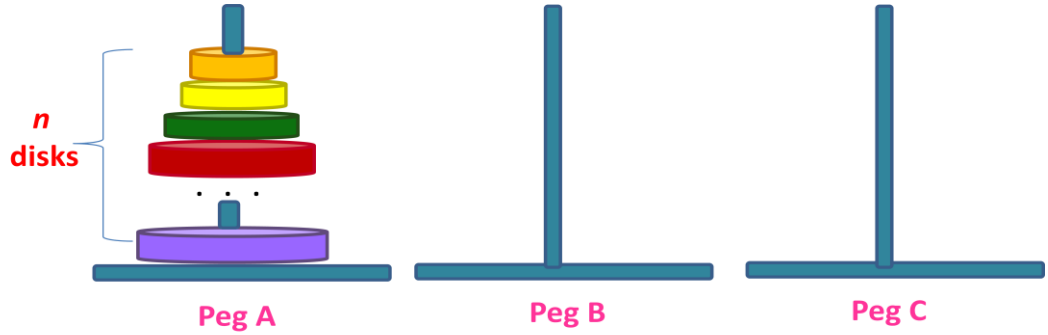
In main() function:

```
res = checkPalindrome(str,0,strlen(str)-1);
```

```
//Recursive Function
int checkPalindrome(char  str[],int i,int j)
{
       if(i>=j)  //Base Case for success
           return(1);
       if(str[i] != str[j])   //Base Case for failure
           return(-1);
       return(checkPalindrome(str,i+1,j-1));   //General Case
}
```

Towers of Hanoi

Initial Setup for Towers of Hanoi



- Three pegs are available named Peg A, Peg B and Peg C.
- n disks of varying diameter are placed on Peg A such that smaller disks are placed over larger disks.
- **Problem:** n disks are to be transferred from source Peg A to destination Peg C using Peg B as auxiliary.

Rules for transferring the disks:

- Only one disk can be transferred at a time from any peg to any other peg.
- Larger disk can never be placed over the smaller disk.

Note: If n is the number of disks then the ***minimum (ideal) number of moves*** required to transfer n disks from source to destination is $2^n - 1$.

Examples: If $n = 1$ then, no. of moves required = $2^1-1 = 1$

If $n = 2$ then, no. of moves required = $2^2-1 = 3$

If $n = 3$ then, no. of moves required = $2^3-1 = 7$

If $n = 4$ then, no. of moves required = $2^4-1 = 15$

Recursive Solution for Towers of Hanoi Problem:

Base Case:

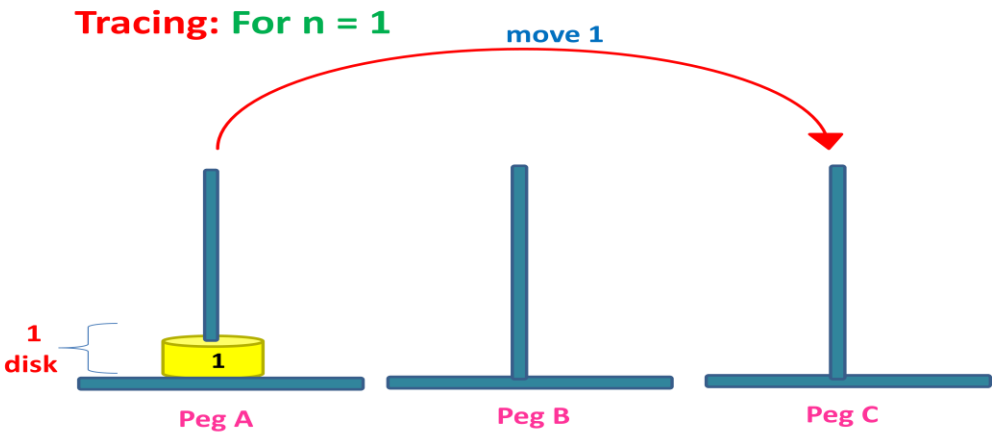
If the number of disks is 1, then transfer the disk from Peg A to Peg C.

General Case:

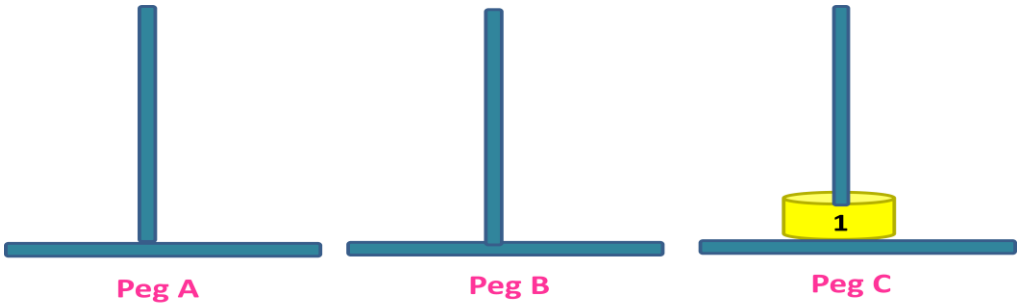
- Recursively transfer *n*-1 disks from Peg A to Peg B using Peg C as auxiliary.
- Transfer the n^{th} disk from Peg A to Peg C.
- Recursively transfer *n*-1 disks from Peg B to Peg C using Peg A as auxiliary.

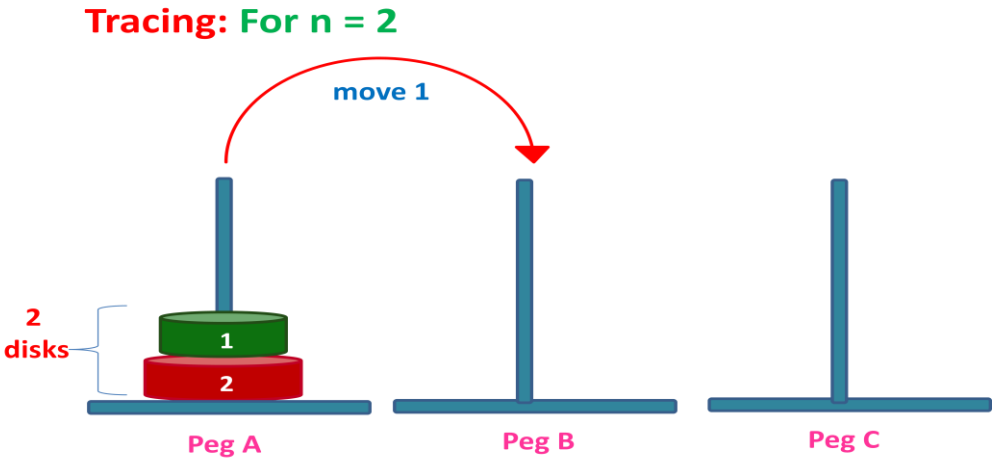
C Program for Towers of Hanoi Problem

```
#include<stdio.h>
int moves;
void TOH(int n,char src,char temp,char dest)
{
     if(n == 1)
    {
          printf("\nTransfer disk %d from Peg %c to Peg %c",n,src,dest);
          moves++;
          return;
     }

     TOH(n-1,src,dest,temp);

     printf("\nTransfer disk %d from Peg %c to Peg %c",n,src,dest);
     moves++;

     TOH(n-1,temp,src,dest);
}

int main()
{
     int n;
     printf("\nEnter the number of disks: ");
     scanf("%d",&n);

     TOH(n,'A','B','C');

     printf("\nTotal number of moves taken = %d",moves);

     return 0;
}
```

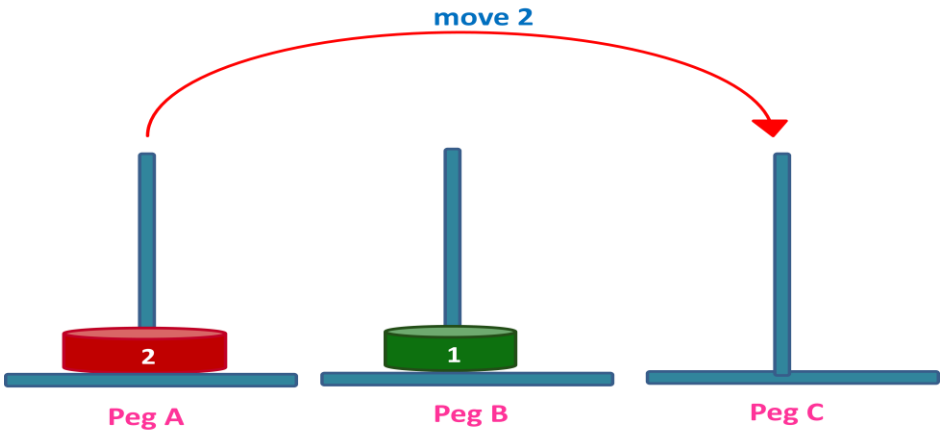


Transfer disk 1 from Peg A to Peg C

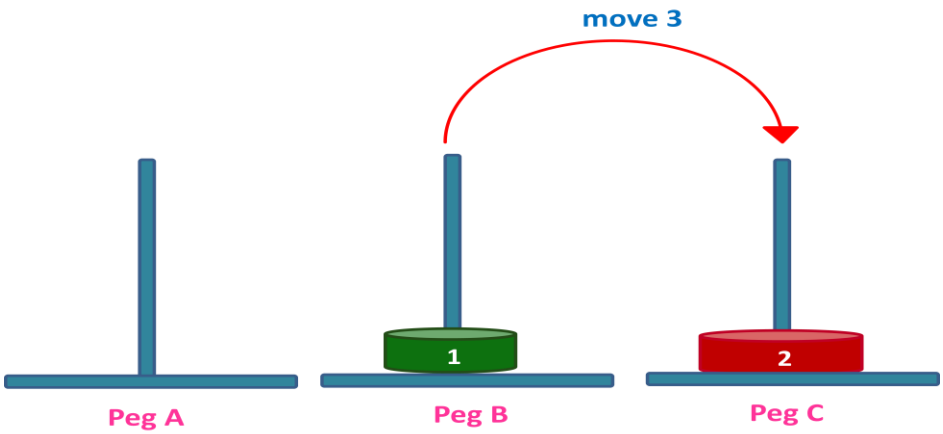




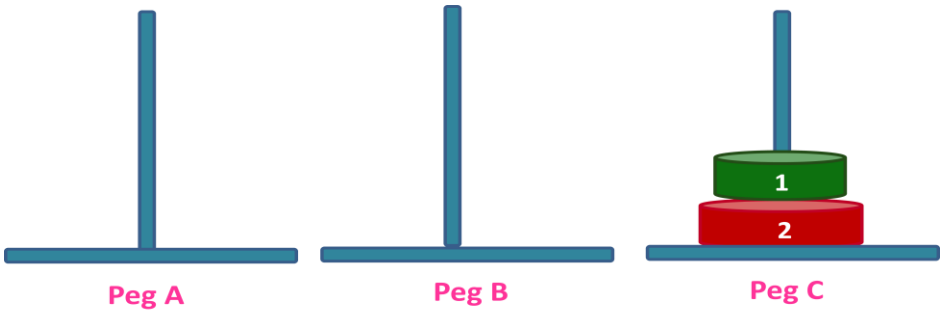
Transfer disk 1 from Peg A to Peg B

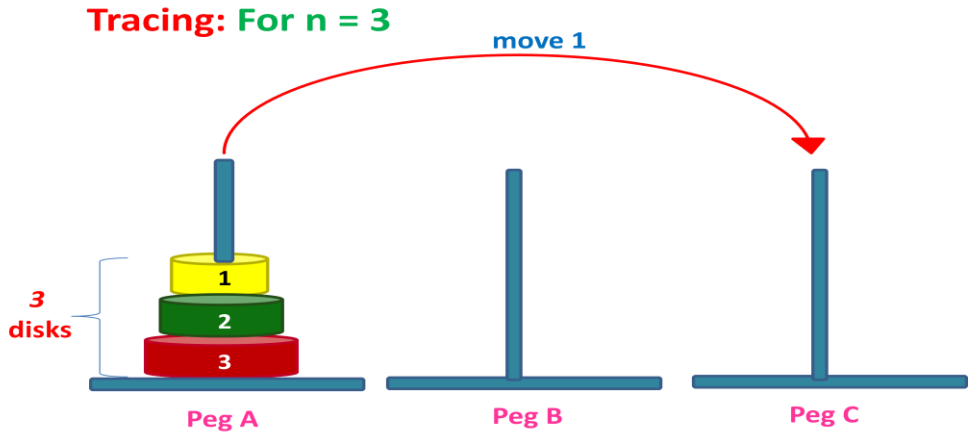


Transfer disk 2 from Peg A to Peg C

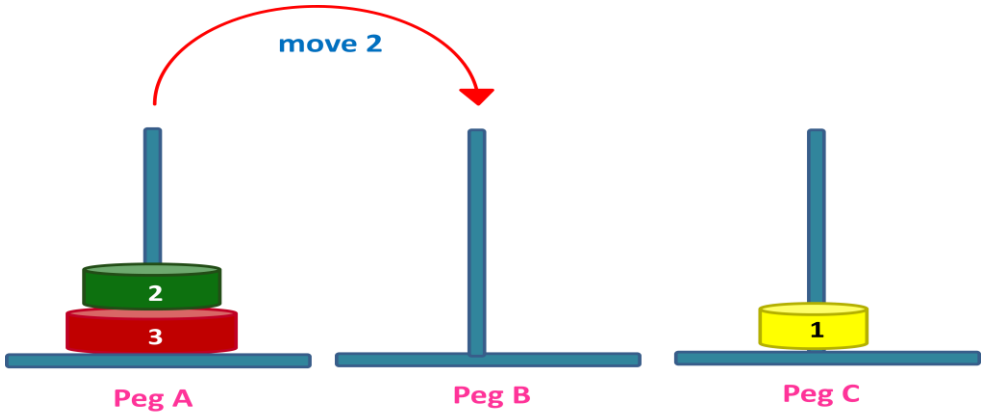


Transfer disk 1 from Peg B to Peg C

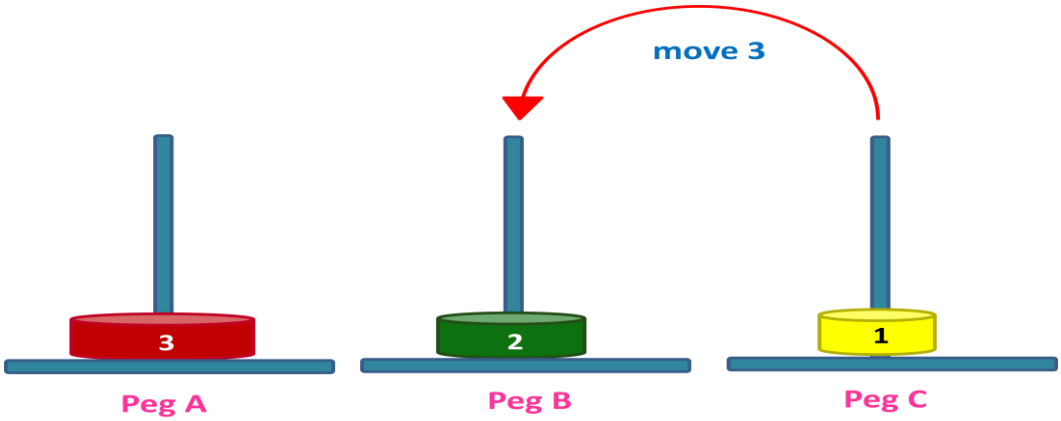




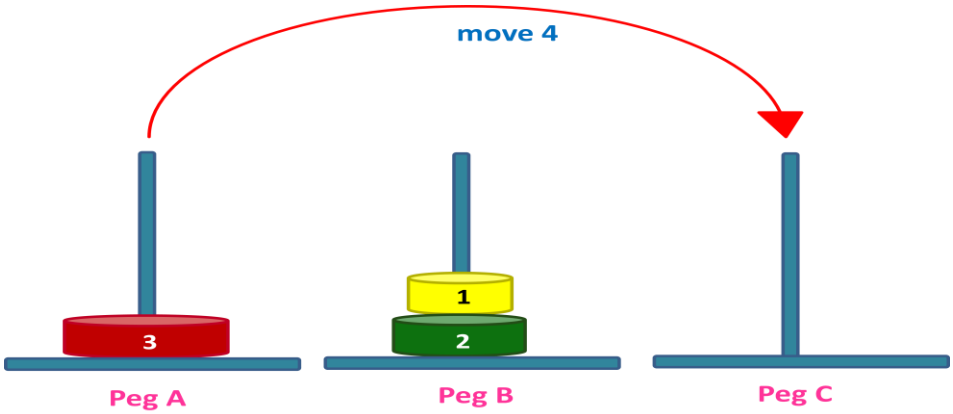
Transfer disk 1 from Peg A to Peg C



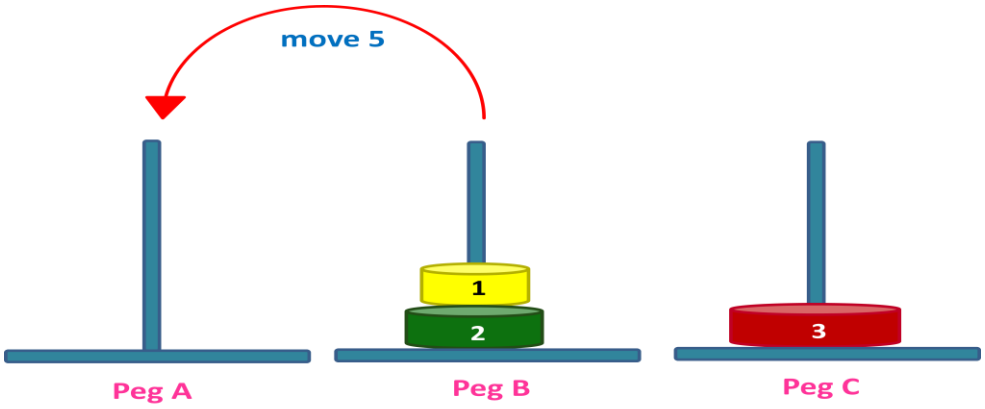
Transfer disk 2 from Peg A to Peg B



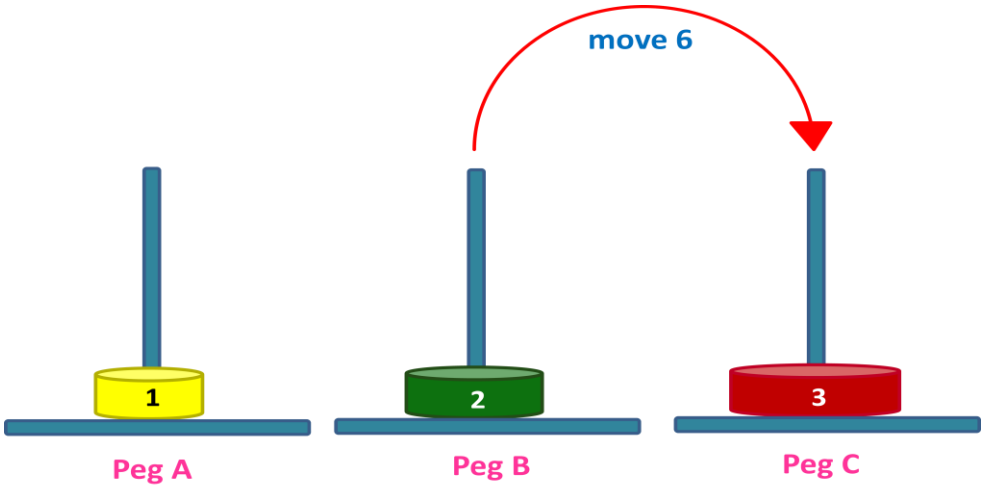
Transfer disk 1 from Peg C to Peg B



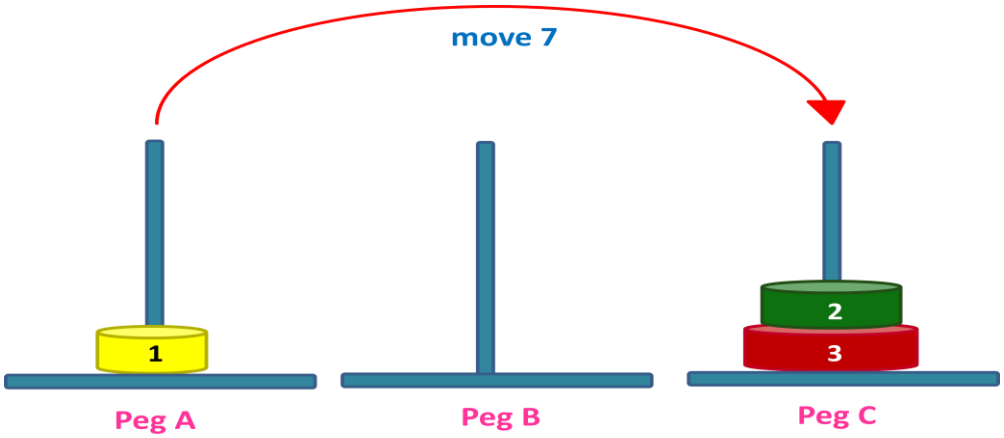
Transfer disk 3 from Peg A to Peg C



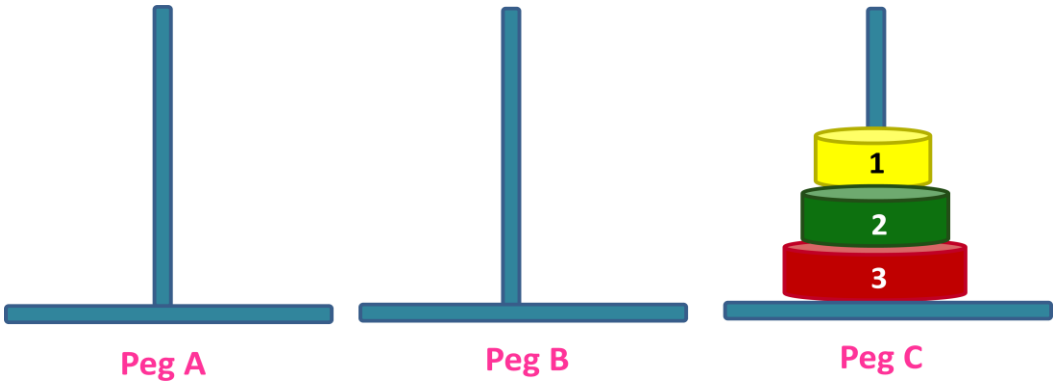
Transfer disk 1 from Peg B to Peg A



Transfer disk 2 from Peg B to Peg C



Transfer disk 1 from Peg A to Peg C



QUEUE

- **Definition:** It is a non-primitive linear data structure in which a new element can be inserted at one end called the **rear end** and an element can be deleted from the other end called the **front end**.
- Queue is also called as **FIFO** (First In First Out) data structure since the first element inserted will be the first to be removed from the Queue.

Representation of Queue in C:

```
#define MAXSIZE 4
typedef struct
{
        int items[MAXSIZE];
        int f,r;
}QUEUE;
```

Types of Queues:

- Linear Queue/Ordinary Queue
- Circular Queue
- Priority Queue
- Double Ended Queue

LINEAR QUEUE

Insertion Operation on Queue:

- We can insert a new element at the rear end of the Queue.
- If the Queue is full and an attempt is made to insert an element to the Queue then it results in a situation called "**Queue Overflow**" !!!
- Condition to test for "Queue Overflow" : **if(r == MAXSIZE-1)**
  If this condition is true then it indicates that queue is full.

- Assume, MAXSIZE is 3 and the data to be inserted are **10, 5, 3, 7**

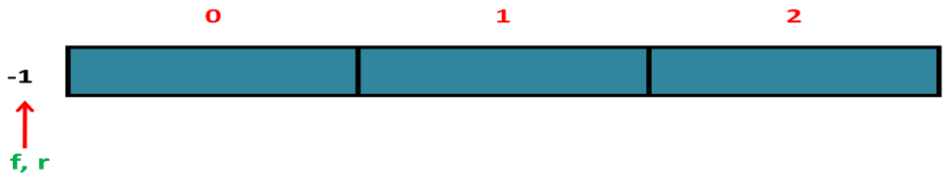


Figure1: Initial Queue (Empty Queue).

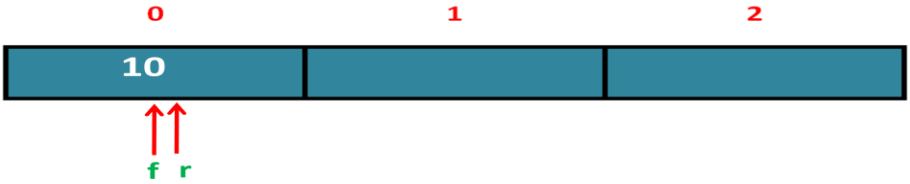


Figure2: After inserting 10.

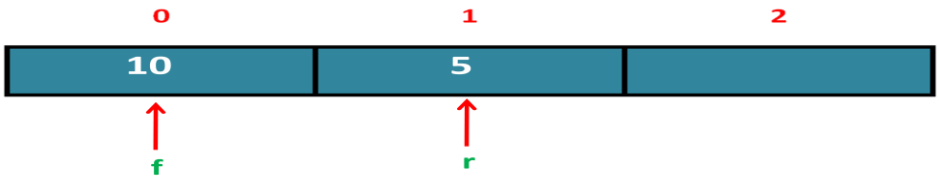


Figure3: After inserting 5.

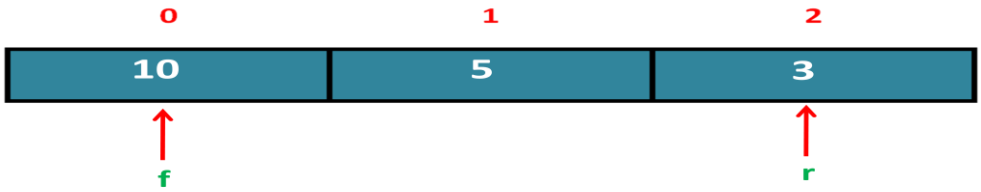


Figure4: After inserting 3.

Since, Queue is full, data 7 cannot be inserted. This operation has resulted in Queue Overflow.

Deletion Operation on Linear Queue:

- We can remove an element at the front end of the queue.
- If the Queue is empty and an attempt is made to delete an element from the queue then it results in a situation called "**Queue underflow**" !!!
- Condition to test for "Queue underflow" : **if(f == -1)**

If this condition is true then it indicates that ***Queue is empty***.

- Assume, MAXSIZE is 3

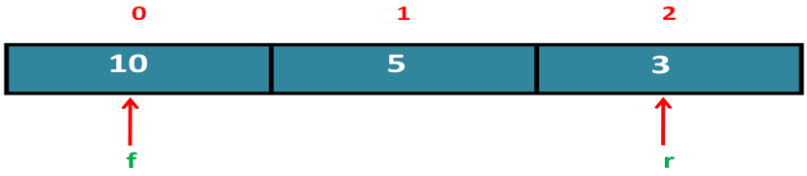


Figure1: Current Queue Status.

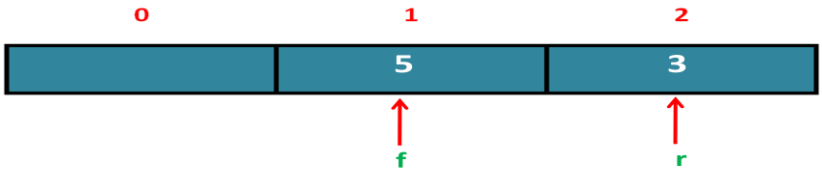


Figure2: After deleting 10.

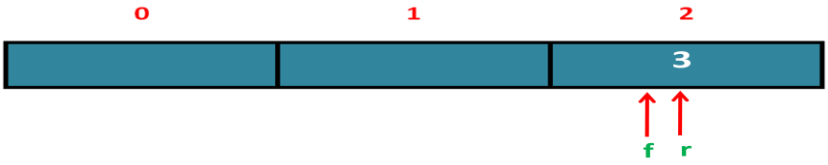


Figure3: After deleting 5.

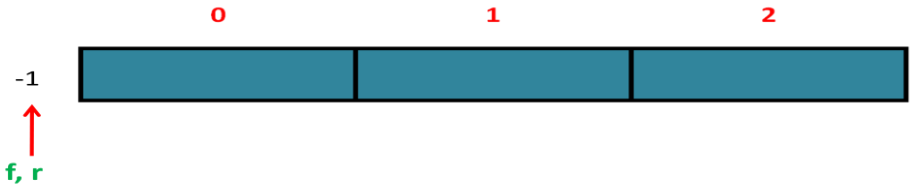


Figure4: After deleting 3.

Display Operation on Linear Queue:

- Display operation means listing the contents of the Queue either from front end of the queue till rear end or from rear end of the queue till front end of the queue.
- Check for queue empty: **if(f == -1) printf("Queue is Empty");**
- Print the Queue elements one by one:

```
for(i=f;i<=r;i++)
    printf("%d->",items[i]);
```

Lab Program5: ***Develop a C program to implement Linear Queue of characters to perform the insertion, deletion and display operations.***

```
#include<stdio.h>
#include<stdlib.h>
#define MAXSIZE  3

typedef struct
{
        char items[MAXSIZE];
        int f,r;
}QUEUE;

int isfull(QUEUE q)
{
        if(q.r == MAXSIZE-1)
             return 1;
        return 0;
}

int isempty(QUEUE q)
{
       if(q.f == -1)
            return 1;
       return 0;
}

void INSERT(QUEUE *q,char data)
{
        q->items[++q->r]=data;
        printf("\nCharacter \'%c\' is inserted into queue",data);
        if(q->f==-1)
             q->f=0;
}

char DELETE(QUEUE *q)
{
         char data;
         data = q->items[q->f];
         if(q->f == q->r)
               q->f = q->r = -1;
         else
               q->f++;
         return(data);
}

void DISPLAY(QUEUE q)
{
         int i;
```

```
        printf("\nQUEUE CONTENTS:\nFRONT->");
        for(i=q.f;i<=q.r;i++)
              printf("%c->",q.items[i]);
        printf("REAR");
}

int main()
{
       QUEUE q;
       int choice;
       char data;

       q.f=q.r=-1;

       while(1)
      {
            printf("\n\n1:Insert\n2:Delete\n3:Display\n4:Exit");
            printf("\nEnter your choice: ");
            scanf("%d",&choice);
            switch(choice)
           {
                   case 1: if(isfull(q))
                                 printf("\nQueue Overflow !!!");
                           else
                           {
                                 printf("\nEnter the character to be inserted: ");
                                 getchar();
                                 scanf("%c",&data);
                                 INSERT(&q,data);
                           }
                           break;

                    case 2: if(isempty(q))
                                  printf("\nQueue Underflow !!!");
                            else
                                 printf("\nCharacter \'%c\' is deleted from queue",DELETE(&q));
                            break;

                   case 3: if(isempty(q))
                                printf("\nQueue is Empty !!!");
                           else
                                DISPLAY(q);
                           break;

                    case 4:  exit(0);
                   default: printf("\nInvalid choice");
              }
        }
        return 0;
}
```

Limitations of Linear Queue:

- In Linear queue, *f* and *r* show linear motion.
- Once *r* reaches MAXSIZE-1, it is not possible to insert an element even though there is free memory space.
  This limitation can be overcome using Circular Queue.
- In circular queue, *f* and *r* show circular motion.

Insertion Operation on Circular Queue:

- The statement used to update *r* is as follows:

  r = (r+1)%MAXSIZE;

- Assume, MAXSIZE is 3 and the data to be inserted are 10, 20, 30, 40

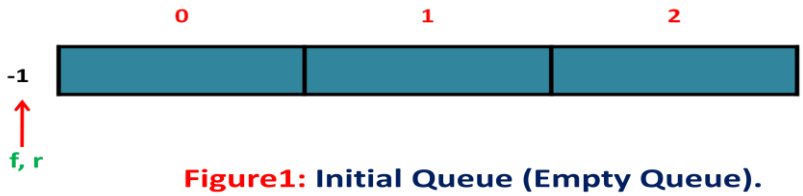


Figure1: Initial Queue (Empty Queue).

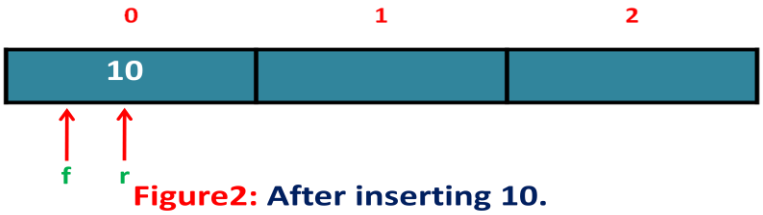


Figure2: After inserting 10.

r = (r+1)%MAXSIZE = (-1+1)%3 = 0%3 = 0.

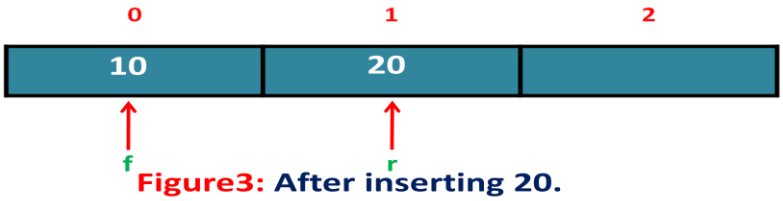


Figure3: After inserting 20.

r = (r+1)%MAXSIZE = (0+1)%3 = 1%3 = 1.

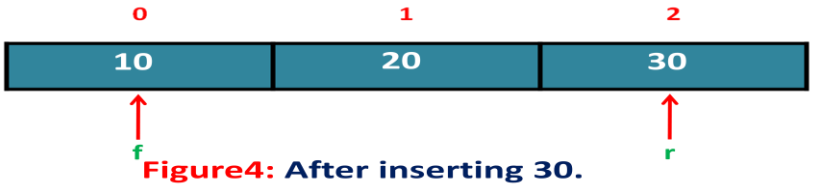


Figure4: After inserting 30.

r = (r+1)%MAXSIZE = (1+1)%3 = 2%3 = 2.

Since queue is full, data 40 cannot be inserted. This operation has resulted in Queue Overflow.

- Condition to check for Circular Queue overflow:

```
if(((f==0) && (r==MAXSIZE-1)) || (f==(r+1)))
        printf("Circular Queue Overflow");
                or
if(f == (r+1)%MAXSIZE)
        printf("Circular Queue Overflow");
```

Deletion Operation on Circular Queue:

- The statement used to update *f* is as follows:

```
f = (f+1)%MAXSIZE;
```

- Consider the Queue of size 3:

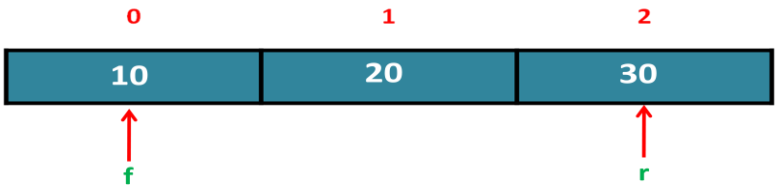


Figure1: Current Queue Status.

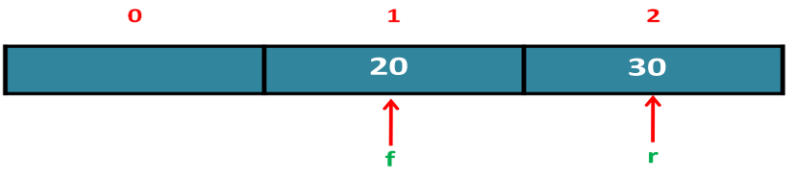


Figure2: After deleting 10.

f = (f+1)%MAXSIZE = (0+1)%3 = 1%3 = 1.

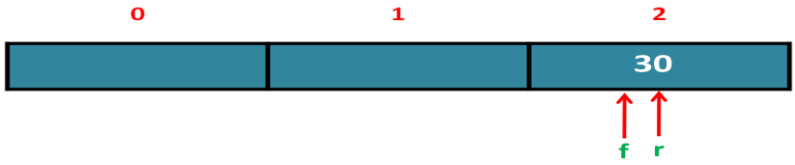


Figure3: After deleting 20.

f = (f+1)%MAXSIZE = (1+1)%3 = 2%3 = 2.

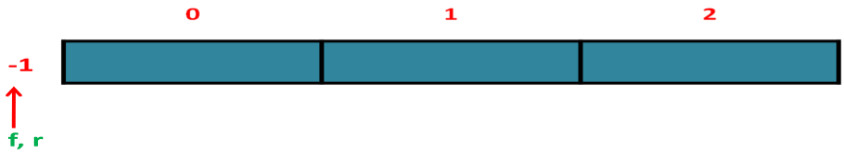


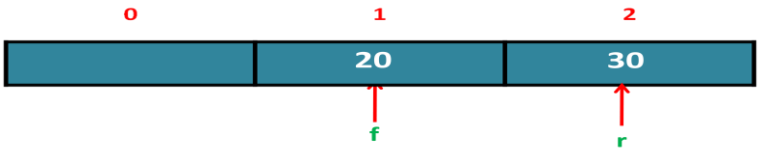
Figure4: After deleting 30.

- Condition to check for Circular Queue underflow:

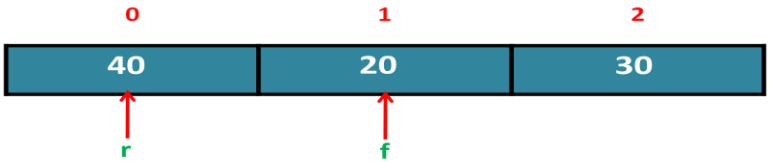
```
if(f == -1)
    printf("Circular Queue Underflow");
```

Display Operation on Circular Queue:

- **If *f* <= *r* then elements can be displayed from *f* to *r*.**



- **If *f* > *r* then elements can be displayed from *f* to MAXSIZE-1 and then from 0 to *r*.**



Lab Program6: ***Develop a C program to implement Circular Queue of integers to perform the insertion, deletion and display operations.***

```
#include<stdio.h>
#include<stdlib.h>
#define MAXSIZE  3
int count;

typedef struct
{
        int items[MAXSIZE];
        int f,r;
}QUEUE;

int isfull(QUEUE q)
{
        if(q.f == (q.r+1)%MAXSIZE)
             return 1;
        return 0;
}

int isempty(QUEUE q)
{
       if(q.f == -1)
            return 1;
       return 0;
}

void INSERT(QUEUE *q,int data)
{
        q->r=(q->r+1)%MAXSIZE;
        q->items[q->r]=data;
        printf("\n%d is inserted into circular queue",data);
        count++;
        if(q->f==-1)
              q->f=0;
}

int DELETE(QUEUE *q)
{
         int data;
         data = q->items[q->f];
         count--;
         if(q->f == q->r)
              q->f = q->r = -1;
         else
              q->f = (q->f +1)%MAXSIZE;
         return(data);
}
```

```
void DISPLAY(QUEUE q)
{
        int i;
        printf("\nQUEUE CONTENTS:\nFRONT->");
        for(i=1;i<=count;i++)
       {
              printf("%d->",q.items[q.f]);
             q.f=(q.f+1)%MAXSIZE;
       }
        printf("REAR");
}

int main()
{
      QUEUE q;
      int choice;
      int data;
      q.f=q.r=-1;
      while(1)
     {
          printf("\n\n1:Insert\n2:Delete\n3:Display\n4:Exit");
          printf("\nEnter your choice: ");
          scanf("%d",&choice);
          switch(choice)
         {
                case 1: if(isfull(q))
                              printf("\nCircular Queue Overflow !!!");
                        else
                        {
                              printf("\nEnter the data to be inserted: ");
                              scanf("%d",&data);
                              INSERT(&q,data);
                        }
                        break;

                 case 2: if(isempty(q))
                              printf("\nCircular Queue Underflow !!!");
                         else
                              printf("\n%d is deleted from queue",DELETE(&q));
                         break;

                 case 3: if(isempty(q))
                              printf("\nCircular Queue is Empty !!!");
                         else
                              DISPLAY(q);
                         break;

                 case 4:  exit(0);
                 default: printf("\nInvalid choice");
```

```
            }
        }
        return 0;
}
```

Problem: Consider a circular queue of size 5 with following elements: 25, 4, 1 where, F=2 and R = 4. Insert 50 and 12 into the queue and delete one element. Write the queue status indicating the positions of F and R after each operation.

Solution:

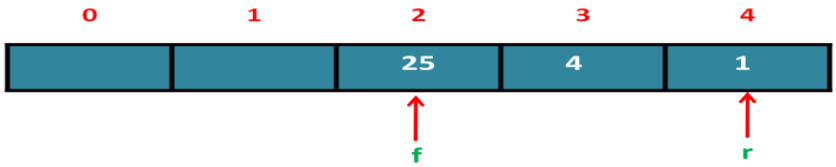


Figure1: Initial Queue status

Given, F = 2 and R = 4

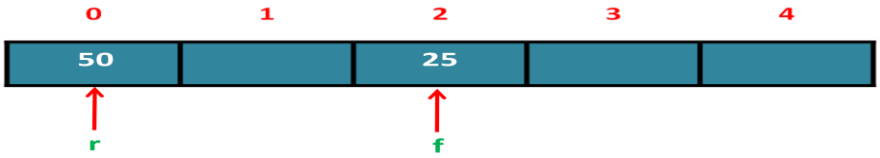


Figure2: After Inserting 50.

After inserting 50, F = 2 and R = 0

r = (r+1)%MAXSIZE = (0+1)%5 = 1%5 = 1.

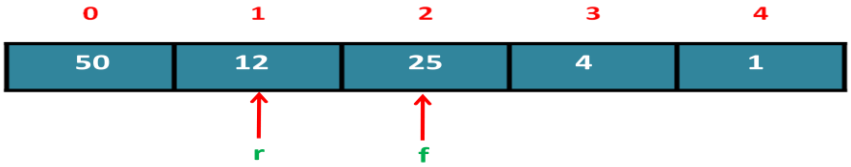


Figure3: After Inserting 12.

After inserting 12, F = 2 and R = 1

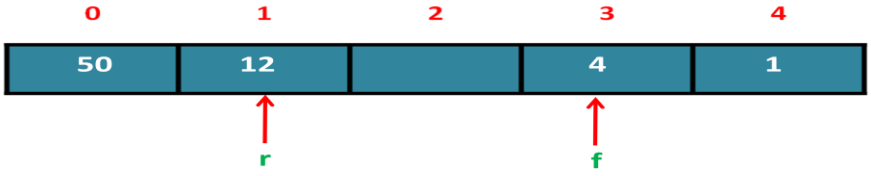


Figure4: After deleting 25.

After deleting 25, F = 3 and R = 1

Double Ended Queue (Dequeue)

Definition: It is a non-primitive linear data structure in which insertion and deletion operations can be performed at both the ends of the queue.

Figure: *Double Ended Queue.*

Variants of Dequeue:

- Input-Restricted Dequeue
- Output-Restricted Dequeue

Input-Restricted Dequeue

Definition: It is a non-primitive linear data structure in which *deletion* operation can be performed at both the ends of the queue but *insertion* operation can be performed at only one end of the queue.

Figure (a): *Input-Restricted Dequeue (Insertion at rear end and deletion at both the ends).*

Figure (b): *Input-Restricted Dequeue (Insertion at front end and deletion at both the ends).*

Output-Restricted Dequeue

Definition: It is a non-primitive linear data structure in which *insertion* operation can be performed at both the ends of the queue but *deletion* operation can be performed at only one end of the queue.

Figure (a): *Output-Restricted Dequeue (Deletion at front end and insertion at both the ends).*

Figure (b): *Output-Restricted Dequeue (Deletion at rear end and insertion at both the ends).*

Priority Queue

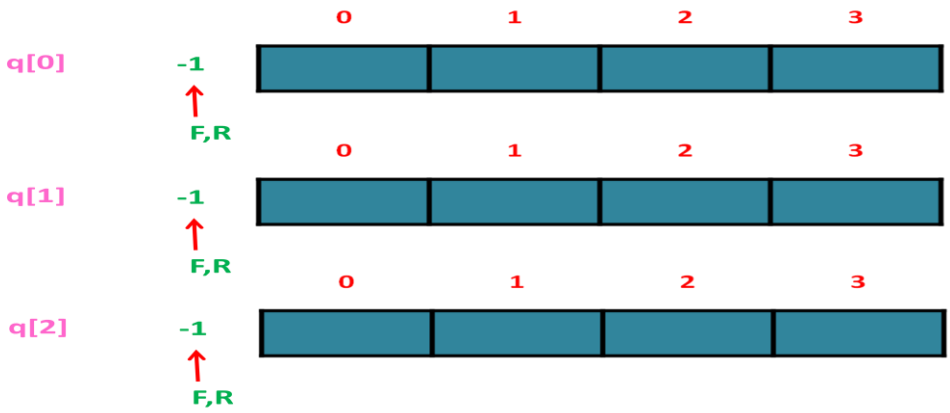
Definition: It is a non-primitive linear data structure in which the elements are inserted or deleted based on some priority.

Priority Queue Implementation

- Using Multiple Queues
- Using Single Queue

Implementation of Priority Queue using Multiple Queues

- Assume that the no. of queues are 3 and priority of queue0, queue1 and queue2 are 0, 1 and 2 respectively.



- Multiple queues are used to perform insertion and deletion operations based on priority.
- Each queue exhibits FIFO behaviour.

- Each queue has its own pair of ***f*** and ***r***.

Insertion Operation:

- The elements are inserted into the appropriate queue based on the priority.

Deletion Operation:

- An element from queue0 is deleted first. Once queue0 becomes empty, element from queue1 is deleted and so on.

Implementation of Priority Queue using Single Queue

- There are two types of priority queues:
  - Ascending Priority Queue
  - Descending Priority Queue

Ascending Priority Queue

- It is a non-primitive linear data structure into which element can be inserted in any arbitrary order but while deleting, ***smallest*** element is deleted first.

Descending Priority Queue

- It is a non-primitive linear data structure into which element can be inserted in any arbitrary order but while deleting, ***largest*** element is deleted first.

********** **END OF UNIT - 2** **********